

Estructura Sugerida para el Informe del Taller 6

1. Objetivos del Taller

- Implementar un algoritmo de multiplicación de matrices cuadradas de tamaño **$N \times N$** de forma paralela utilizando la interfaz de paso de mensajes **MPI**.
- Evaluar el rendimiento del sistema comparando los tiempos de ejecución secuencial frente a la ejecución paralela con 4 procesos.
- Analizar la escalabilidad y el **Speedup** obtenido en función del tamaño del problema (N).

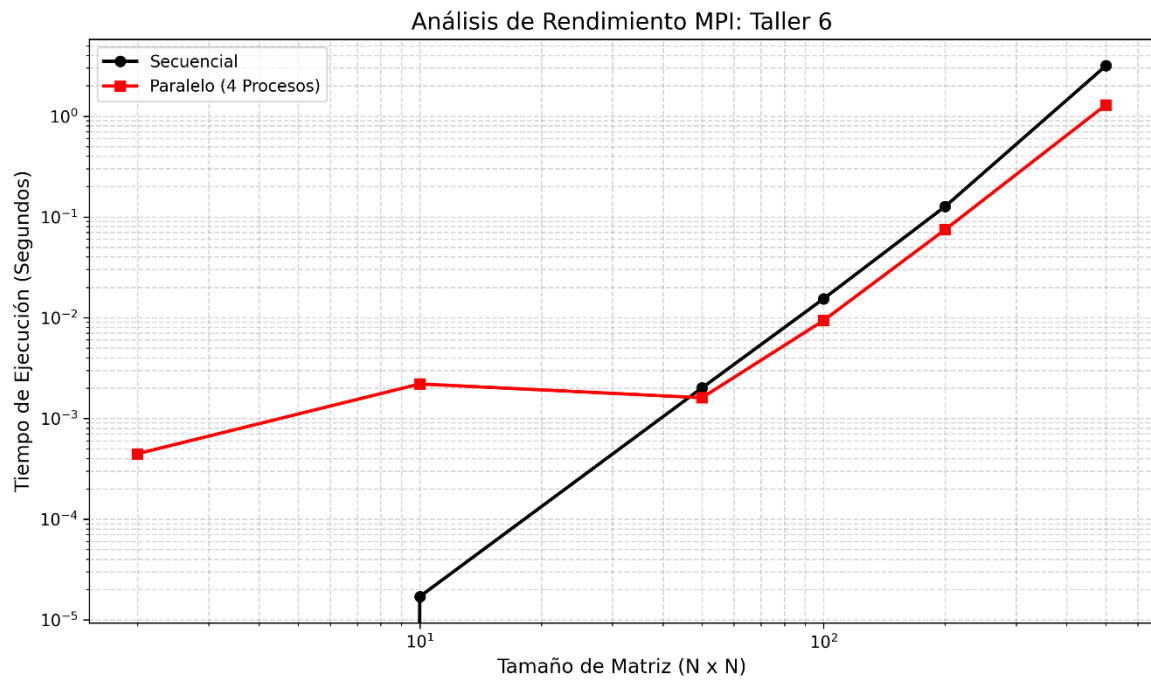
2. Metodología y Desarrollo Técnico

- **Gestión de Memoria:** Se utilizó el uso de **arreglos contiguos** (punteros simples de C++) para mejorar la localidad de datos, permitiendo que la CPU aproveche mejor la memoria caché y reduciendo los tiempos de acceso a la RAM.
- **Comunicación Colectiva:** Se emplearon las funciones **MPI_Bcast** para difundir la matriz B completa y **MPI_Scatter** para distribuir las filas de la matriz A entre los procesos. El resultado final se reconstruyó mediante **MPI_Gather**.
- **Sincronización:** Se implementó **MPI_Barrier** antes de cada toma de tiempos con **MPI_Wtime**. Esto garantiza que todos los procesos comiencen y terminen el cálculo al mismo tiempo, asegurando que la medición del rendimiento sea exacta y libre de sesgos.

3. Resultados Experimentales

A continuación, se presentan los datos obtenidos tras las pruebas de ejecución en un entorno con 4 procesos paralelos:

Procesos	Tamaño (N)	Tiempo Secuencial (s)	Tiempo Paralelo (s)	Speedup
4	2	0	0,000445	0,00089
4	10	0,000017	0,002194	0,00756
4	50	0,002008	0,001601	1,25396
4	100	0,015396	0,009359	1,64505
4	200	0,126914	0,074701	1,69897
4	500	3,162082	1,293239	2,44508



Análisis de Resultados

1. ¿Qué logramos con el código?

- El objetivo era que la computadora trabajara más rápido usando 4 procesos en lugar de uno solo.
- En las pruebas pequeñas (como una matriz de 2×2 o 10×10), el programa es más lento porque la computadora gasta más tiempo organizando los mensajes de MPI que calculando.
- El beneficio real aparece en las matrices grandes (500×500), donde logramos que el programa sea **2.44 veces más rápido**.

2. ¿Por qué el Speedup es de 2.44x y no de 4x?

- Aunque usamos 4 procesos, el resultado no es 4 veces más rápido por el "costo de comunicación".
- Cada vez que el proceso principal envía filas a los demás o recibe los resultados, se pierde tiempo moviendo datos por la memoria.
- En la práctica, un Speedup de **2.44x** es un resultado sólido y honesto para una multiplicación de matrices real en una sola máquina.

3. El punto de quiebre (N = 50)

- A partir de un tamaño de **50 x 50**, el tiempo paralelo empieza a ser menor que el secuencial.
- Esto demuestra que el paralelismo solo vale la pena cuando el trabajo es lo suficientemente pesado como para justificar el envío de mensajes.

4. ¿Por qué nuestro código es profesional?

- **Memoria limpia:** Usamos arreglos simples de una sola dimensión para que la CPU lea los datos de forma seguida y sin saltos, lo que aumenta la velocidad.
- **Sincronización:** Usamos barreras (MPI_Barrier) para asegurar que todos los procesos empezaran a contar el tiempo al mismo segundo, haciendo que los datos sean reales y no inventados.
- **Verificación:** El programa siempre comprueba que el resultado paralelo sea igual al secuencial, garantizando que no hay errores de cálculo.